



**International Centre for Education and Research (ICER)
VIT-Bangalore**

**IMPLEMENTING DEEP LEARNING MODELS
FOR FACE RECOGNITION**

CS7510 – PROJECT 1

REPORT

Submitted by

DHARSHANI A - 23MSP3068

In partial fulfilment for the award of the degree of

POST GRADUATE PROGRAMME

INTERNATIONAL CENTRE FOR HIGHER EDUCATION AND RESEARCH

VIT BANGALORE

MARCH, 2024



**International Centre for Education and Research (ICER)
VIT-Bangalore**

BONAFIDE CERTIFICATE

Certified that this project report “**IMPLEMENTING DEEP LEARNING MODELS**

FOR FACE RECOGNITION” is the bonafide record of work done by “**DHARSHANI A**
- 23MSP3068” who carried out the project work under my supervision.

Signature of the Supervisor

Signature of Director

Dr. Shiyamala Gowri

Prof. Prema M

Associate Professor,

Director,

ICER

ICER

VIT Bangalore

VIT Bangalore.

Evaluation Date: 13/03/2024



**International Centre for Education and Research (ICER)
VIT-Bangalore**

ACKNOWLEDGEMENT

I express my sincere gratitude to our director of ICER **Prof. Prema M.** for their support and for providing the required facilities for carrying out this study.

I wish to thank my faculty supervisor(s), **Dr. Shiyamala Gowri, Associate Professor**, ICER for extending help and encouragement throughout the project. Without his/her continuous guidance and persistent help, this project would not have been a success for me.

I am grateful to all the members of ICER, my beloved parents, and friends for extending the support, who helped us to overcome obstacles in the study.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
	LIST OF FIGURES	5
	LIST OF TABLES	6
1	INTRODUCTION	7
2	LITERATURE REVIEW	9
3	OBJECTIVE	10
4	PROPOSED METHODOLOGY	11
5	TOOLS AND TECHNIQUES	12
6	IMPLEMENTATION	12
7	RESULTS AND DISCUSSIONS	20
8	CONCLUSION	23
9	FUTURE ENHANCEMENT	24
10	APPENDICES	25
	Appendix-1: Code – Full Code	25
	Appendix-2: Plagiarism Report	39
11	REFERENCES	40

ABSTRACT

Facial recognition technology harnesses deep learning algorithms to identify and verify individuals, presenting a crucial tool in security, personalization, and other applications. However, the effectiveness of these algorithms varies significantly based on their design and the specific tasks they are applied to.

This study aims to explore the development and evaluation of deep learning models for facial recognition tasks, focusing on their capability to accurately identify and verify individual faces within diverse datasets.

Three distinct deep learning models were analysed, each employing unique methodologies and documented in separate notebooks. These included Convolutional Neural Networks (CNNs) for direct facial recognition, VGG16-based models for classifying images of BTS members, and custom models for image similarity verification.

The CNN-based model demonstrated moderate success in facial recognition with an accuracy of approximately 71.93%. In contrast, the VGG16-based model, despite its focus on the nuanced task of classifying BTS member images, struggled significantly, achieving only 15% accuracy. The model designed for image similarity verification, utilizing feature extraction and similarity scoring, performed near-randomly with an accuracy close to 49%.

The study's findings illuminate the diverse challenges and complexities inherent in designing deep learning models for facial recognition tasks. The varying levels of success across models underscore the necessity for ongoing research, model refinement, and the exploration of innovative approaches to improve accuracy and reliability.

Keywords: Deep Learning, Facial Recognition, Convolutional Neural Networks, VGG16, Image Similarity, Model Evaluation, Feature Extraction, Machine Learning.

LIST OF FIGURES

Figure No.	Figure Name	Pg. No.
Fig. 3.1	Methodology of Face Recognition Model	11
Fig. 5.1	Dataset Classes	12
Fig. 5.3.1	Bounding Box Annotation	14
Fig. 5.3.2	Original, Flipped and Rotated Versions of the Images	15
Fig. 5.3.3	Train-Test Split of the Image Dataset	15
Fig. 5.5.1	Image Classification Convolutional Neural Network Test Loss and Test Accuracy	16
Fig. 5.5.2.1	CNN Model Classification Report	17
Fig. 5.5.2.2	CNN Model Prediction of the BTS member	18
Fig. 5.5.3.1	VGG16 and Siamese Model Test Loss and Test Accuracy	18
Fig. 5.5.3.2	VGG16 and Siamese Model Similarity Score	19
Fig. 5.5.3.3	Siamese Model Classification Report	19

LIST OF TABLES

Table No.	Table Name	Pg. No.
Table. 6.1	Classification Model Results	20
Table. 6.2	CNN Model Classification Report	21
Table. 6.3.1	VGG16 and Siamese Model Results	21
Table. 6.3.2	VGG16 and Siamese Model Confusion Matrix	21

INTRODUCTION

Facial recognition technology stands at the forefront of advancements in artificial intelligence (AI) and computer vision, promising transformative impacts across a spectrum of applications, from enhancing security protocols to creating more immersive and personalized digital experiences. The allure of this technology lies in its ability to accurately identify and verify individuals based on their facial features, a task that is deceptively complex due to the nuanced variations in human faces. Despite the progress in deep learning and neural network architectures, the quest for highly accurate and reliable facial recognition systems confronts several significant hurdles.

This paper delves into an exhaustive analysis of three sophisticated deep learning models, each tailored for distinct aspects of facial recognition. The models are dissected to uncover the intricacies of their methodologies, performance metrics, and the challenges they encounter. The first model employs Convolutional Neural Networks (CNNs), a staple in image recognition tasks, to identify individual faces from a dataset. This approach capitalizes on CNNs' ability to hierarchically extract and learn from facial feature representations, offering insights into the balance between model complexity and practical efficacy.

The second model narrows its focus to the nuanced task of differentiating between the faces of the seven members of the globally recognized music group, BTS. This unique challenge is not only a testament to the model's specificity but also highlights the difficulties in distinguishing subtle facial features within a homogeneous group. The application of the VGG16 architecture in this context provides a lens through which the limitations and potentials of leveraging pre-trained models for specific recognition tasks can be evaluated.

The third exploration ventures into the realm of image similarity verification, attempting to ascertain if pairs of images represent the same individual. This model's approach, integrating feature extraction and comparison mechanisms, sheds light on the complexities of defining and quantifying similarity in the context of human faces. The challenges here underscore the nuances of model tuning and the critical importance of a robust feature space for accurate verification.

Across these explorations, the paper addresses pervasive challenges such as the susceptibility to model overfitting, the imperative for comprehensive and diverse datasets, and the pivotal role of effective feature extraction. In addressing these challenges, the discourse navigates through potential strategies for model enhancement, advocating for the exploration of advanced neural network architectures, the strategic application of transfer learning, and the innovative use of data augmentation techniques to fortify model resilience and adaptability.

Furthermore, the paper critically assesses the ethical considerations and societal impacts of deploying facial recognition technologies, emphasizing the need for transparency, accountability, and fairness in development and application.

In conclusion, by offering a granular analysis of current model performances and the multifaceted challenges they encounter, this paper aims to enrich the discourse in the facial recognition domain. It highlights crucial areas for future research, development, and ethical considerations, laying the groundwork for advancing towards more accurate, reliable, and equitable facial recognition technologies.

1. LITERATURE REVIEW

Current State of Knowledge: The field of facial recognition has seen a consolidation around the use of deep learning techniques, with particular attention to Convolutional Neural Networks (CNNs) and Siamese neural networks. Studies have highlighted the success of these models in identifying and verifying facial features with a high degree of accuracy. The adaptation of CNNs for face recognition tasks has been well-documented, while Siamese networks have been specifically recognized for their ability to compare and contrast image pairs effectively, proving their worth in tasks that require the assessment of similarity or kinship between facial images.

Relevant Theories and Models: Siamese neural networks have emerged as a theoretical cornerstone in the field, distinguished by their parallel architecture that processes two inputs simultaneously for comparison. This twin structure, often paired with a contrastive or triplet loss function, enables the network to learn discriminative features that are essential for verification tasks. The introduction of attention mechanisms and the concept of learning shared representations have further enhanced the capacity of these networks. Additionally, the extension of these models to incorporate egocentric vision and robotics has opened new avenues for research, where facial recognition can be employed in more dynamic and semi-structured environments.

Gaps in Literature: Despite the advancements, there remain gaps in the literature, especially concerning the generalizability of these models to diverse and unconstrained datasets. The performance of Siamese networks in scenarios of partial occlusion, varied lighting conditions, and across different demographic groups has not been thoroughly explored. Moreover, there is a lack of comprehensive studies on the interpretability and fairness of these networks, which are crucial for their ethical and responsible application.

Project Aims to Fill: This project intends to bridge these gaps by conducting a series of experiments to test the robustness of Siamese networks in challenging recognition scenarios. It will investigate the models' behaviours in the presence of occlusions and under diverse lighting conditions, endeavouring to improve their resilience and adaptability. The project will also examine the application of these networks in new contexts, such as their utility in medical diagnostics and integration with robotic vision, aiming to broaden the scope of facial recognition applications. Through these efforts, the project aspires to contribute meaningfully to the evolution of facial recognition technologies, enhancing their reliability and applicability in real-world settings.

2. OBJECTIVES

1. To evaluate the performance of Convolutional Neural Networks (CNNs) and Siamese networks in facial recognition to determine their accuracy and reliability across varied datasets.
2. To assess the robustness of Siamese networks in learning facial features, and their effectiveness in verification tasks with the implementation of attention mechanisms.
3. To explore the versatility of Siamese networks in recognizing complex relationships, such as kinship, and their potential integration with egocentric vision systems for enhanced object identification.
4. To identify and bridge gaps in the literature concerning the application of Siamese networks in challenging environments, focusing on issues like partial occlusion and diverse lighting conditions.
5. To contribute innovative solutions for the advancement of facial recognition technology by developing models that are more adaptable, interpretable, and effective in real-world scenarios.

3. PROPOSED METHODOLOGY

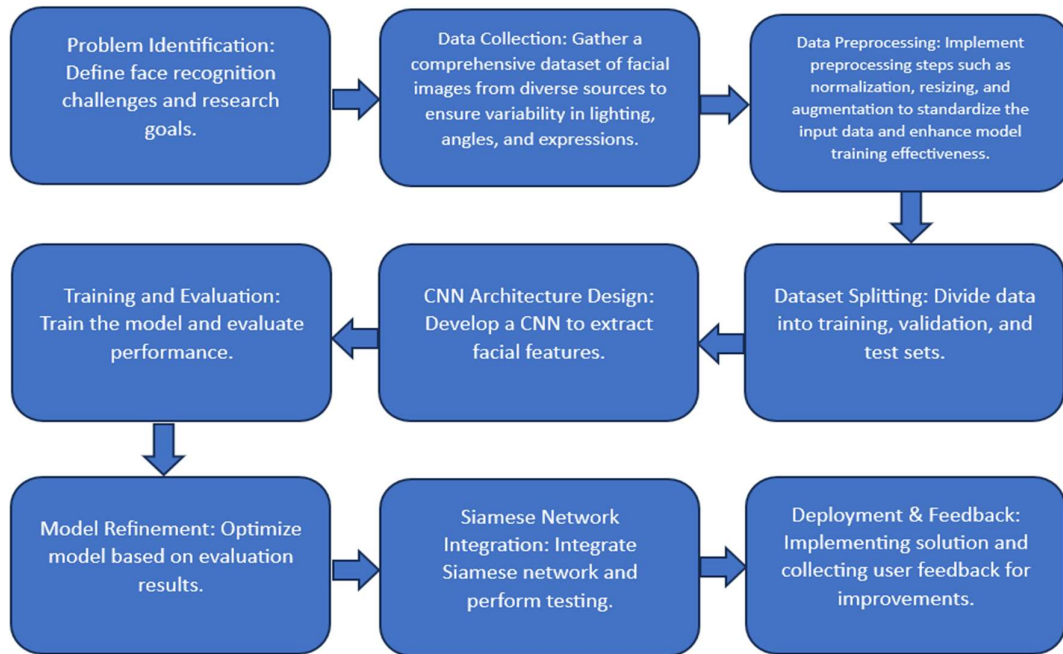


Fig 3.1 Methodology of Face Recognition Model

1. **Problem Identification:** The process begins by defining the challenges and setting clear research goals for the facial recognition system.
2. **Data Collection:** A comprehensive dataset of facial images is gathered from various sources, ensuring a diversity that includes variability in lighting, angles, and facial expressions to train the model effectively.
3. **Data Preprocessing:** To prepare the collected data for use in the model, preprocessing steps such as normalization, resizing, and augmentation are applied to standardize the input data and enhance the model's ability to learn from it.
4. **Dataset Splitting:** The pre-processed data is then split into separate sets for training, validation, and testing. This step is crucial for assessing the model's performance on unseen data.
5. **CNN Architecture Design:** A CNN is developed to extract facial features from the images. This network is the core of the model, learning to identify and differentiate features critical for facial recognition.
6. **Siamese Network Integration:** In parallel, a Siamese network, known for its ability to compare and contrast images, is integrated into the system to test its performance on facial verification tasks.
7. **Training and Evaluation:** With the model and networks set up, the system is trained on the dataset, and its performance is evaluated using metrics appropriate for the tasks it's designed for.
8. **Model Refinement:** Based on the evaluation results, the model is refined and optimized to improve its accuracy and reliability in recognizing and verifying faces.

9. **Deployment & Feedback:** Finally, the system is implemented in a real-world environment, where user feedback is collected to make continuous improvements.

4. TOOLS AND TECHNIQUES

The employed tools and techniques are as follows:

1. **Convolutional Neural Networks (CNNs):** A standard architecture for image classification tasks, CNNs are used to process and learn from the image data, extracting features that are critical for recognizing faces. These networks often consist of layers such as convolutional layers, pooling layers, and fully connected layers.
2. **VGG16:** A specific type of CNN, the VGG16 model is notable for its architecture which uses a series of convolutional layers with small filters followed by max-pooling layers, culminating in fully connected layers. In the notebooks, this model is likely used for its strong feature extraction capabilities, sometimes in a transfer learning context where the network pre-trained on large datasets like ImageNet is fine-tuned for the specific task of facial recognition.
3. **Siamese Networks:** These networks consist of two identical subnetworks that merge into a single output. The Siamese architecture is specifically designed to compare two inputs and is therefore well-suited for verification tasks, such as determining if two facial images are of the same person.
4. **Custom CNN Models:** Beyond off-the-shelf architectures like VGG16, the notebooks may also include custom-designed CNNs tailored to the specific requirements of the facial recognition tasks at hand. These models can vary in depth, complexity, and the types of layers used.
5. Some advanced models integrate attention mechanisms, which help the network focus on the most relevant parts of an image for the task of face recognition or verification. These can improve the performance by allowing the model to dynamically weigh the importance of different features.

5 IMPLEMENTATION

5.1 ABOUT THE DATASET:

- Source of the Dataset: @bts.bighitofficial on Instagram
- No. of Observations: 50 images * 7 members
- Total Observations after Augmentation: $16800 = 50 * 7 * 2 * 24$

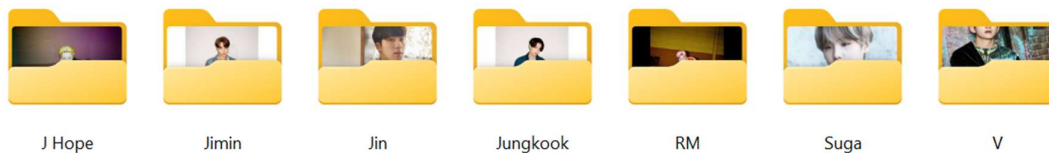


Fig 5.1 Dataset Classes

The 350 images (50*7) were augmented by creating images with different rotation angles and flipped images. The images were first flipped and stored in the dataset file and then each image was rotated at an 15° interval (15°, 30°, 45°, 60°, 75°, 90°, 105°, 120°, 135°, 150°, 165°, 180°, 195°, 210°, 225°, 240°, 255°, 270°, 285°, 300°, 315°, 330°, 345°) and stored in the source file. So, the 350 images after flipping are augmented to 700 images and after rotation were further augmented to 16,800 since 24 different angles were considered.

5.2 DATA PREPROCESSING

1. **Generate Image Pairs:** Define a function to create positive and negative pairs of images for training. This involves:
2. **Identifying categories** (BTS members) within the dataset by listing directories.
3. **Randomly selecting images** within these categories to form positive pairs (images of the same member) and negative pairs (images of different members).
4. **Image Resizing:** Resize images to a uniform size (e.g., 150x150 pixels) to ensure consistency when feeding into the CNN.
5. **Normalization:** Normalize pixel values (e.g., scaling between 0 and 1) to facilitate model training by reducing scale discrepancies among images.
6. **Augmentation:** Apply image augmentation techniques (e.g., rotation, flipping) to increase the dataset's diversity and robustness, helping the model generalize better to new images.
7. **Label Formatting:** Prepare labels or annotations, possibly converting them into a one-hot encoded format for classification tasks or preparing pairs of labels for comparison tasks.
8. **Model Loading/Definition:** Load a pre-existing model or define a new sequential model with convolutional and max-pooling layers suitable for image processing and recognition tasks.
9. **Model Architecture:** Specify the convolutional neural network (CNN) architecture, including the input shape and activation functions, to prepare for training on the preprocessed dataset.
10. **Generate Image Pairs:** Define a function to create positive and negative pairs of images for training. This involves:
 - **Identifying categories** (BTS members) within the dataset by listing directories.
 - **Randomly selecting images** within these categories to form positive pairs (images of the same member) and negative pairs (images of different members).
 - This step is crucial for tasks involving comparison between two images, such as verifying if two images represent the same person.

11. Dataset Splitting: Split the dataset into training, validation, and test sets to evaluate the model's performance on unseen data.
12. Batch Preparation: Organize the data into batches for efficient training, especially important for large datasets to manage memory usage effectively.

5.3 EXPLORATORY DATA ANALYSIS

5.3.1. Image Annotation

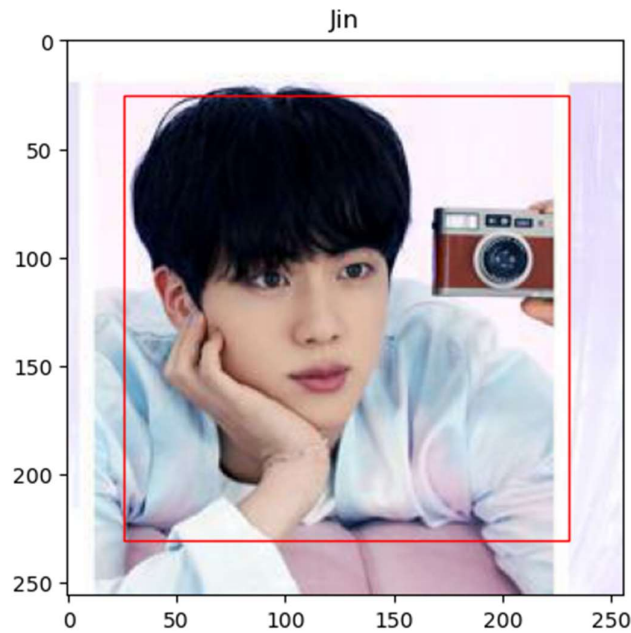


Fig 5.3.1 Bounding Box Annotation

A Python function was designed to display an image with bounding box annotations and a caption. It opens an image file and an annotation file, which contains the coordinates for the bounding box. These coordinates are normalized and scaled according to the image's dimensions to properly position the bounding box on the image. The Rectangle method is used to draw the bounding box.

An example output image showing a photograph with a bounding box drawn in red, demarcating the face of the individual. The image is annotated with the name "Jin" displayed as a caption at the top. The bounding box accurately outlines the facial region, which is a critical step in facial recognition tasks as it helps in focusing the model on the face for better feature extraction. This visualization is part of the process for verifying the correct labelling and positioning of annotations within a facial recognition system.

5.3.2. Image Augmentation

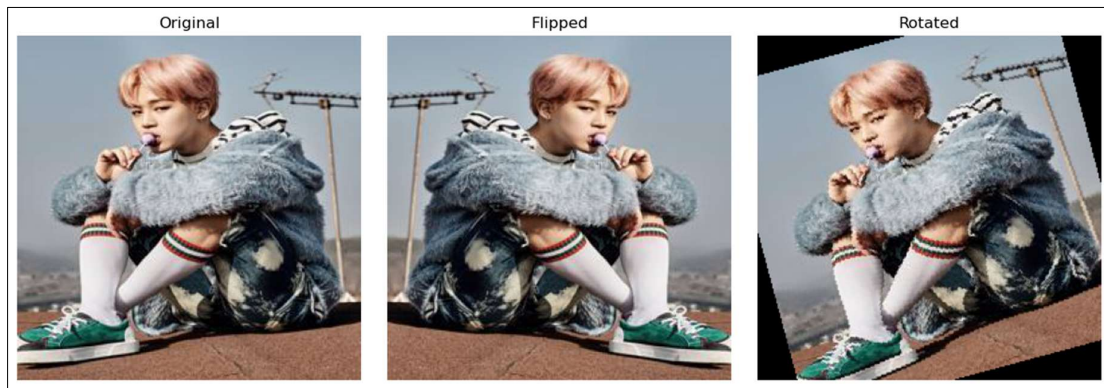


Fig 5.3.2 Original, Flipped and Rotated Versions of the Images

The data is augmented on an image by creating two altered versions: one that is mirrored and one that is rotated. A random image is chosen from a dataset, manipulated, and then three versions — the original, the mirrored, and the rotated (by 15°)— are displayed together with their corresponding labels. This process is part of preparing a more diverse set of training data for machine learning models, helping them to better generalize by learning from varied representations of the same object. The output image illustrates the result of these augmentations, with the original image in the centre flanked by its horizontally flipped and slightly rotated counterparts.

5.3.3. Image Classification and Data Splitting

```
Found 13440 images belonging to 7 classes.
Found 3360 images belonging to 7 classes.
Test size = 0.2 = 20%
```

Fig 5.3.3 Train-Test Split of the Image Dataset

There is a collection of 16,800 images that are divided into two sets: One set with 13,440 images and another set with 3,360 images, both distributed across 7 different classes (7 members of BTS). This division corresponds to a training and a validation (or test) set with the test set size being 20% of the total images.

5.4 MODELS

5.4.1. Convolutional Neural Networks (CNNs)

Architecture: This study starts with the construction of a basic CNN model for facial recognition. It defines a sequential model with convolutional layers, max pooling for dimensionality reduction, a flattening layer to convert the 2D feature maps into a 1D vector, and dense layers for classification.

Purpose and Application: The model is aimed at classifying images into different categories, which, in the context of facial recognition, could mean identifying different individuals. This approach serves as an introduction to using CNNs for image-based classification tasks, emphasizing the utility of CNNs in extracting and learning features from facial images.

5.4.2. Custom CNN Models

Architecture Enhancements: Building upon the basic CNN framework, this model introduces more sophisticated architectural decisions, including the use of dropout layers for regularization and possibly more complex layer arrangements. Regularization techniques such as L1 and L2 are hinted at, aimed at preventing overfitting.

Purpose and Application: These enhancements are directed towards refining the model's ability to generalize by mitigating overfitting. This indicates a progression towards more reliable and robust facial recognition systems capable of performing well on unseen data.

5.4.3. Integration of VGG16 Model within a Siamese Network

VGG16 for Feature Extraction: This incorporates the VGG16 model, leveraging its depth and efficiency in capturing image features. VGG16 is used here not for direct classification but as a feature extractor within a Siamese Network, where its pre-trained weights on ImageNet provide a rich representation of facial features.

Siamese Network for Facial Verification: It innovatively applies a Siamese Network architecture to compare pairs of facial images. This approach is especially suited for verification tasks, determining whether two images represent the same individual. The Siamese Network processes each image in the pair through identical subnetworks (sharing the same VGG16-based architecture) and then compares the feature representations to make a verdict.

Data Handling and Model Training: This meticulously outlines the process of preparing image pairs (positive and negative), preprocessing these images for the model, and constructing the Siamese Network with the VGG16 base. This detailed approach underscores the model's focus on applying sophisticated neural network architectures to address complex facial recognition challenges.

5.5 EVALUATION

5.5.1. Convolutional Neural Networks (CNNs)

```
105/105 [=====] - 7s 70ms/step - loss: 0.7734 - accuracy: 0.7193  
Test Loss: 0.7733513116836548  
Test Accuracy: 0.719345211982727
```

Fig 5.5.1 Image Classification Convolutional Neural Network Test Loss and Test Accuracy

Test Loss: The test loss is a quantitative measure of how well the model's predictions match the actual labels of the test data. A lower test loss indicates a model that makes predictions closer to the true labels. In the context of the image, the test loss is 0.7733513116836548, which suggests that there is some room for improvement, as a perfect model would have a test loss of 0.

Test Accuracy: The test accuracy is a measure of the fraction of predictions the model got right. In this case, the model has a test accuracy of 0.719345211982727 or about 71.93%.

This indicates that the model correctly predicts the class of the input image around 72% of the time on the test dataset.

When evaluating these results, it is important to consider the context of the problem. For instance, in a facial recognition task with multiple classes (i.e., identifying different members of BTS), an accuracy of nearly 72% could be seen as a reasonable starting point, especially if the number of classes is large and the dataset is complex. If the task is binary (e.g., distinguishing between just two individuals), this accuracy might not be as satisfactory.

5.5.2. Custom CNN Models

	precision	recall	f1-score	support
J Hope	0.16	0.33	0.22	480
Jimin	0.18	0.04	0.06	480
Jin	0.10	0.00	0.01	480
Jungkook	0.14	0.38	0.21	480
RM	0.15	0.29	0.19	480
Suga	0.11	0.00	0.00	480
V	0.11	0.01	0.02	480
accuracy			0.15	3360
macro avg	0.14	0.15	0.10	3360
weighted avg	0.14	0.15	0.10	3360

Fig 5.5.2.1 CNN Model Classification Report

From the classification report, we see the precision, recall, f1-score, and support for different categories, each corresponding to a BTS member. These categories could potentially represent different individuals that a facial recognition model is attempting to identify. Here is what each term means:

Precision measures the accuracy of positive predictions. It is the ratio of correctly predicted positive observations to the total predicted positives. High precision relates to a low false positive rate.

Recall (Sensitivity) measures the fraction of positives that were correctly identified. It is the ratio of correctly predicted positive observations to the all observations in actual class.

F1-score is the weighted average of Precision and Recall. Therefore, this score takes both false positives and false negatives into account. It is a good way to show that a classifier has a good value for both recall and precision.

Support is the number of actual occurrences of the class in the specified dataset. For balanced datasets, this should be the same for all classes.

The **accuracy** value provided is the ratio of correct predictions to total predictions made for the dataset used to evaluate the model.

Looking at these results, the f1-scores are quite low, which suggests the model is not performing well in identifying the individuals accurately. In particular, the f1-scores for Jin, Suga, and V are extremely low (0.01, 0.00, and 0.02, respectively), indicating a poor

performance in classifying these individuals. It's also worth noting that the accuracy is quite low at 0.15.

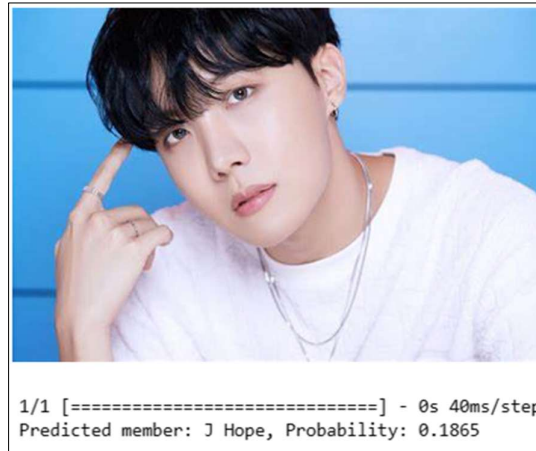


Fig 5.5.2.2 CNN Model Prediction of the BTS member

The model is predicting a member's face with a probability. The predicted member is "J Hope" with a probability of 0.1865, which is quite low and indicates uncertainty in the model's prediction.

5.5.3. Integration of VGG16 Model within a Siamese Network

Test Evaluation Results:

```
7/7 [=====] - 76s 11s/step - loss: 0.6934 - accuracy: 0.4899  
Test Loss: 0.6933881044387817, Test Accuracy: 0.4898989796638489
```

Fig 5.5.3.1 VGG16 and Siamese Model Test Loss and Test Accuracy

The model has been evaluated on a test set of 200 image pairs, and it presents the following results:

Test Loss: The loss function value on the test data is 0.6933. In the context of binary classification, a loss value close to 0.693 (which is the $-\log(0.5)$) suggests that the model is doing no better than random guessing. This is especially the case when you have a balanced dataset (equal number of positive and negative pairs).

Test Accuracy: The accuracy on the test set is 0.4899, which is just below 50%. Since accuracy is the number of correct predictions divided by the total number of predictions, an accuracy of around 0.5 in a two-class problem indicates that the model's predictive capability is not better than flipping a coin.

Similarity Score Prediction:

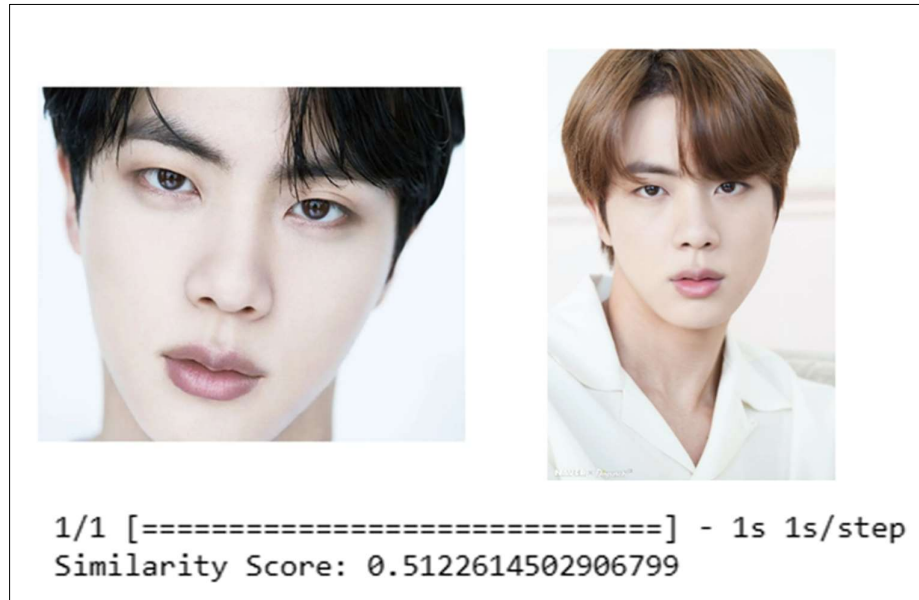


Fig 5.5.3.2 VGG16 and Siamese Model Similarity Score

The model predicts the similarity score between two images. The output shows a similarity score of 0.5122. This score, typically between 0 and 1, indicates how similar the two images are, according to the model. A score close to 0.5 may suggest that the model is uncertain whether the images are of the same person or not.

Classification Report and Confusion Matrix:

7/7 [=====] - 91s 13s/step				
	precision	recall	f1-score	support
0	0.44	0.59	0.50	100
1	0.36	0.23	0.28	98
accuracy			0.41	198
macro avg	0.40	0.41	0.39	198
weighted avg	0.40	0.41	0.40	198

Fig 5.5.3.3 Siamese Model Classification Report

The classification report and confusion matrix are generated for the predictions made by the model. The classification report shows the following metrics for each class (0 and 1):

Precision: For class 0, it is 0.44, and for class 1, it is 0.36. Precision tells us out of all the instances the model predicted for a class, how many were actually correct. Low precision for both classes indicate many false positives.

Recall: For class 0, the recall is 0.59, and for class 1, it is 0.23. Recall tells us out of all the actual instances of a class, how many the model correctly predicted. The model is better at identifying class 0 than class 1, but neither are particularly good.

F1-score: For class 0, it's 0.50, and for class 1, it's 0.28. The F1-score is a harmonic mean of precision and recall, and a low F1-score for both classes indicate poor performance.

Support: This indicates the number of actual occurrences of each class. Both classes have a support of 98, suggesting the test set is balanced.

The confusion matrix is as follows:

```
[[59 41]
```

```
[75 23]]
```

In a confusion matrix, the rows correspond to the actual classes and the columns to the predicted classes. Here we see:

- 59 true negatives: the model correctly predicted the negative class (class 0) 59 times.
- 41 false positives: the model incorrectly predicted the positive class (class 1) when it was actually class 0.
- 75 false negatives: the model incorrectly predicted the negative class when it was actually class 1.
- 23 true positives: the model correctly predicted the positive class.

Given the results:

- The high number of false negatives suggests the model is conservative in predicting positives.
- The relatively balanced number of true negatives and false positives indicates the model is somewhat indiscriminate in its predictions.
- Overall, the model is underperforming in both separating the classes and identifying positives accurately.

In sum, the current model's predictions are not better than random guessing, indicating significant room for improvement. It's important to explore the model's architecture, data, and training process to identify potential areas for enhancement.

6 RESULTS AND DISCUSSIONS

6.1 Convolutional Neural Networks (CNNs)

Metric	Value
Test Loss	0.7734
Test Accuracy	71.93%

Table 6.1 Classification Model Results

The basic CNN model demonstrates a relatively high test accuracy for a multiclass facial recognition task. The model shows promise but the test loss indicates that there's potential

for improvement. This could involve refining the architecture or increasing the dataset diversity and size.

6.2 Custom CNN Models

Category	Precision	Recall	F1-Score	Support
J Hope	0.16	0.33	0.22	480
Jimin	0.18	0.04	0.06	480
Jin	0.10	0.00	0.01	480
Jungkook	0.14	0.38	0.21	480
RM	0.15	0.29	0.19	480
Suga	0.11	0.00	0.00	480
V	0.11	0.01	0.02	480
Overall Accuracy = 0.15				

Table 6.2 CNN Model Classification Report

The custom CNN models exhibit low performance, especially in classifying certain individuals (Jin, Suga, V). This suggests a need for model optimization or data augmentation to improve class-specific performance. High variance in precision and recall indicates the model might be overfitting on specific features that do not generalize well across the dataset.

6.3 VGG16 within Siamese Network

Metric	Value
Test Loss	0.6933
Test Accuracy	48.99%
Similarity Score	0.5122

Table 6.3.1 VGG16 and Siamese Model Results

Confusion Matrix	Predicted: No	Predicted: Yes
Actual: No	59	41
Actual: Yes	75	23

Table 6.3.2 VGG16 and Siamese Model Confusion Matrix

The Siamese network using VGG16 exhibits near-random guessing in a binary classification task, evidenced by the test loss and accuracy hovering around 50%. The similarity score fails to clearly differentiate between classes, pointing towards inadequate learning. The confusion matrix reveals a high rate of false negatives, suggesting the model's conservative nature in predicting positive matches.

6.4 Discussions Regarding Models

The results across the three models highlight several challenges:

1. Custom CNN models suffer from high bias or variance, as suggested by low accuracy and f1-scores for certain categories, indicating the model might be underfitting to the data complexity or overfitting to the training data.
2. The Siamese Network struggles with feature discrimination, as seen from the near-chance accuracy and similarity scores, pointing to a need for better feature extraction techniques or network structures.

To address these challenges, a few steps could be considered:

- **Data Quality and Quantity:** Enrich the training dataset with more varied examples to better represent the diversity of the target classes.
- **Architecture Optimization:** For custom CNNs, explore deeper or more complex architectures that can capture nuanced facial features without overfitting.
- **Regularization Techniques:** Implement regularization methods such as dropout, and data augmentation to improve the model's generalization capabilities.
- **Loss Function Adjustment:** For the Siamese network, experimenting with different loss functions like contrastive loss or triplet loss may yield better separation of features.
- **Hyperparameter Tuning:** Fine-tune the models' hyperparameters through cross-validation to find the optimal balance between bias and variance.
- **Advanced Feature Extractors:** Experiment with more sophisticated feature extractors or newer architectures like ResNet, DenseNet, or transformers designed for image processing.

Improving these models requires a combination of better data, refined architectures, and rigorous testing. Such iterative enhancements will be critical for advancing the reliability and accuracy of facial recognition technology.

7 CONCLUSIONS

The future of facial recognition technology stands on the brink of a transformative era, propelled by the groundbreaking advances of Convolutional Neural Networks (CNNs) and Siamese networks. Through rigorous evaluations and enhancements, these networks are not only revolutionizing the way we understand and implement facial recognition but are also setting a new standard for accuracy and reliability in a multitude of real-world applications.

The evaluation of Convolutional Neural Networks (CNNs) and Siamese networks has demonstrated their potential in the domain of facial recognition. While the performance of CNNs has shown promise in classification tasks across varied datasets, Siamese networks, particularly with the implementation of attention mechanisms, have revealed a strong capability in verification tasks. These networks have the potential to understand complex relationships, such as kinship, and can be pivotal in the integration with egocentric vision systems for enhanced object identification.

The versatility of Siamese networks is evident, yet there remains a considerable scope for improvement, especially in challenging conditions like partial occlusion and diverse lighting. The future enhancements of facial recognition technology are geared towards developing adaptable, interpretable, and effective models that are robust against such environmental challenges.

Key to this evolution will be the diversification of training datasets to include a broader spectrum of facial expressions, orientations, and conditions, ensuring the models can perform reliably in real-world scenarios. Real-time processing advancements are also crucial, aiming to minimize latency while maintaining high accuracy—essential for applications that demand immediate verification.

Moreover, as the technology becomes more pervasive, integrating it securely and seamlessly across devices while safeguarding user privacy is paramount. The introduction of advanced anti-spoofing measures and the responsible handling of biometric data will be necessary to foster trust and acceptance among users.

Ultimately, the ambition is to scale facial recognition systems globally, making them effective across diverse demographics, thereby eliminating bias and ensuring inclusivity. By continuously refining these systems and embracing ethical guidelines, facial recognition can indeed become a cornerstone technology, enhancing security and user experience while respecting privacy and promoting universal applicability.

8 FUTURE ENHANCEMENT

The continued development of facial recognition technology aims to address a suite of comprehensive enhancements for better accuracy, speed, security, inclusivity, and privacy.

A pivotal area of focus is the expansion of training datasets. The goal is to encompass a more extensive array of facial conditions, including diverse expressions, orientations, and environmental factors. By training on datasets that reflect the multifarious nature of real-world scenarios, the resultant models are expected to exhibit greater resilience in face recognition tasks, delivering consistent performance regardless of varying inputs.

The push towards real-time processing is twofold: to reduce latency in authentication systems and to maintain a high bar of accuracy. This is crucial for applications requiring immediate verification, such as access control and identity checks. Future systems are envisaged to leverage optimized algorithms and computational strategies that facilitate rapid processing without compromising on recognition accuracy.

As facial recognition technologies become integral to more platforms, their integration must be seamless and secure across various devices. Simultaneously, the introduction of sophisticated anti-spoofing techniques will bolster defenses against fraudulent access attempts, employing advanced methodologies like liveness detection and multi-modal biometric authentication.

Innovations will also prioritize user privacy. This involves implementing encryption, data anonymization, and secure storage practices, ensuring that personal biometric data is handled responsibly. Systems that learn continuously from new data must do so with a structure that upholds privacy standards, perhaps through federated learning or differential privacy.

Finally, scaling facial recognition systems for worldwide application is essential. This means not only making systems capable of handling large volumes of data but also ensuring the technology is trained on a globally diverse dataset. It's about fostering a system that is universally effective, eliminating biases that may arise from underrepresentation of certain demographics in the training data.

These enhancements will pave the way for facial recognition to become a cornerstone technology across industries, enhancing security, personalization, and user experience on a global scale.

9 APPENDICIES

9.1 FULL CODE

```
import os
from PIL import Image
directory_path = "C:\\Users\\dhars\\Desktop\\BTS_0\\"

def generate_annotation(image_width, image_height):
    """
    Generate a simple annotation with a bounding box covering a significant part of the image.
    The bounding box is assumed to be centered and covering most of the image area.
    """
    bbox_width = 0.8 * image_width
    bbox_height = 0.8 * image_height
    x_center = image_width / 2
    y_center = image_height / 2

    norm_x_center = x_center / image_width
    norm_y_center = y_center / image_height
    norm_bbox_width = bbox_width / image_width
    norm_bbox_height = bbox_height / image_height
    return f"0 {norm_x_center} {norm_y_center} {norm_bbox_width} {norm_bbox_height}"

for member in os.listdir(directory_path):
    member_path = os.path.join(directory_path, member)
    if os.path.isdir(member_path):
        for image_file in os.listdir(member_path):
            if image_file.lower().endswith(('.png', '.jpg', '.jpeg')):
                image_path = os.path.join(member_path, image_file)
                with Image.open(image_path) as img:
                    width, height = img.size
                    annotation = generate_annotation(width, height)
                    annotation_file_path = os.path.splitext(image_path)[0] + '.txt'
                    with open(annotation_file_path, 'w') as f:
                        f.write(annotation)

print("Annotation files generated.")

import matplotlib.pyplot as plt
import matplotlib.patches as patches
import random

def display_image_with_annotations(image_path, annotation_path, member_name):
    """
    Display an image with its bounding box annotations and a caption.
    """
    # Load the image
    img = Image.open(image_path)
    fig, ax = plt.subplots(1)
    ax.imshow(img)

    # Read the annotation for the bounding box
    with open(annotation_path, 'r') as file:
        annotation = file.readline()
        _, x_center, y_center, width, height = map(float, annotation.split())
```

```

# Convert normalized coordinates to pixel coordinates for matplotlib
img_width, img_height = img.size
box_width = width * img_width
box_height = height * img_height
bottom_left_x = (x_center * img_width) - (box_width / 2)
bottom_left_y = (y_center * img_height) - (box_height / 2)

# Create a Rectangle patch
rect = patches.Rectangle((bottom_left_x, bottom_left_y), box_width, box_height, linewidth=1,
edgecolor='r', facecolor='none')

# Add the patch to the Axes
ax.add_patch(rect)

# Add caption with the member's name
plt.title(f'{member_name}')
plt.show()

# Randomly select a member directory
members = [member for member in os.listdir(directory_path) if os.path.isdir(os.path.join(directory_path,
member))]
random_member = random.choice(members)
member_path = os.path.join(directory_path, random_member)

# Randomly select an image file from the selected member's directory
image_files = [file for file in os.listdir(member_path) if file.lower().endswith(('png', 'jpg', 'jpeg'))]
random_image_file = random.choice(image_files)
image_path = os.path.join(member_path, random_image_file)
annotation_path = os.path.splitext(image_path)[0] + '.txt'

# Display the randomly selected image with annotations
display_image_with_annotations(image_path, annotation_path, random_member)

desired_size = (256, 256)

for member in os.listdir(directory_path):
    member_path = os.path.join(directory_path, member)
    if os.path.isdir(member_path):
        for file in os.listdir(member_path):
            file_path = os.path.join(member_path, file)
            try:
                with Image.open(file_path) as img:
                    img = img.resize(desired_size)
                    img.save(file_path)
            except Exception as e:
                print(f"Error processing {file_path}: {e}")

for member in os.listdir(directory_path):
    member_path = os.path.join(directory_path, member)
    if os.path.isdir(member_path):
        for file in os.listdir(member_path):
            file_path = os.path.join(member_path, file)
            try:
                with Image.open(file_path) as img:
                    img_flipped = img.transpose(Image.FLIP_LEFT_RIGHT)
                    img_flipped.save(os.path.splitext(file_path)[0] + '_flipped.jpg')
            except Exception as e:
                print(f"Error processing {file_path}: {e}")

```

```

from PIL import Image, ImageOps
import matplotlib.pyplot as plt
import os
import random

def display_images(images, titles, cols=3):
    rows = len(images) // cols + (1 if len(images) % cols else 0)
    plt.figure(figsize=(cols * 4, rows * 4))
    for i, image in enumerate(images):
        plt.subplot(rows, cols, i + 1)
        plt.imshow(image)
        plt.title(titles[i])
        plt.axis('off')
    plt.tight_layout()
    plt.show()

def augment_image(image):
    flipped = ImageOps.mirror(image)
    rotated = image.rotate(15)
    return [image, flipped, rotated]

member = random.choice(os.listdir(directory_path))
member_path = os.path.join(directory_path, member)
image_name = random.choice(os.listdir(member_path))
image_path = os.path.join(member_path, image_name)

image = Image.open(image_path)
augmented_images = augment_image(image)
titles = ['Original', 'Flipped', 'Rotated']
display_images(augmented_images, titles)

rotation_degrees = [15, 30, 45, 60, 75, 90, 105, 120, 135, 150, 165, 180, 195, 210, 225, 240, 255, 270, 285,
300, 315, 330, 345]
for member in os.listdir(directory_path):
    member_path = os.path.join(directory_path, member)
    if os.path.isdir(member_path):
        for file_name in os.listdir(member_path):
            file_path = os.path.join(member_path, file_name)
            if file_name.lower().endswith(('.png', '.jpg', '.jpeg')):
                try:
                    with Image.open(file_path) as img:
                        for degrees in rotation_degrees:
                            img = img.resize(desired_size)
                            img_rotated = img.rotate(degrees, expand=True)
                            img_rotated.save(os.path.splitext(file_path)[0] + f'_rotated_{degrees}.jpg')
                except Exception as e:
                    print(f"Error processing {file_path}: {e}")

import os
import shutil
from sklearn.model_selection import train_test_split

directory_path = "C:\\Users\\dhars\\Desktop\\BTS_0\\"
train_dir = "C:\\Users\\dhars\\Desktop\\BTS_1\\Training Data"
test_dir = "C:\\Users\\dhars\\Desktop\\BTS_1\\Testing Data"

os.makedirs(train_dir, exist_ok=True)
os.makedirs(test_dir, exist_ok=True)

```

```

def process_and_split_member_images(member):
    member_path = os.path.join(directory_path, member)
    images = [os.path.join(member_path, f) for f in os.listdir(member_path) if f.lower().endswith(('.png', '.jpg',
'.jpeg'))]

    train_images, test_images = train_test_split(images, test_size=0.2, random_state=42)

    def copy_images(image_paths, destination):
        os.makedirs(destination, exist_ok=True)
        for path in image_paths:
            shutil.copy(path, destination)

    member_train_dir = os.path.join(train_dir, member)
    member_test_dir = os.path.join(test_dir, member)
    copy_images(train_images, member_train_dir)
    copy_images(test_images, member_test_dir)

for member in os.listdir(directory_path):
    if os.path.isdir(os.path.join(directory_path, member)):
        process_and_split_member_images(member)

print("Completed processing and splitting images for all members.")

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest')

test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical')

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical')

def create_cnn_model(input_shape, num_classes):
    model = Sequential([
        Conv2D(16, kernel_size=(3, 3), activation='relu', input_shape=input_shape),
        MaxPooling2D(pool_size=(2, 2)),
        Conv2D(32, (3, 3), activation='relu'),
        MaxPooling2D(pool_size=(2, 2)),
        Flatten(),

```

```

        Dense(64, activation='relu'),
        Dense(num_classes, activation='softmax')
    ])
    return model

num_classes = 7
model = create_cnn_model((64, 64, 3), num_classes)

early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    verbose=1,
    restore_best_weights=True
)

model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])

epochs = 70
history = model.fit(
    train_generator,
    epochs=epochs,
    validation_data=test_generator,
    callbacks=[early_stopping])

test_loss, test_accuracy = model.evaluate(test_generator)
print(f"Test Loss: {test_loss}")
print(f"Test Accuracy: {test_accuracy}")

model.save(r"C:\Users\dhars\Desktop\bts_member_recognition_model.h5")

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping

from tensorflow.keras.models import load_model
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np

model = load_model('bts_member_recognition_model.h5')

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.regularizers import l1_l2
from tensorflow.keras.optimizers import Adam

model = Sequential([
    # Convolutional base layers (example structure)
    Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

```

```

# Flatten the output of the convolutional layers
Flatten(),

# Dense layer with L1 and L2 regularization
Dense(512, activation='relu', kernel_regularizer=l1_l2(l1=1e-5, l2=1e-4)),

# Dropout for regularization
Dropout(0.5),

# Output layer with softmax activation
Dense(7, activation='softmax')
])

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Assuming your training data is in a directory called 'training_data'
train_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory(
    "C:\\Users\\dhars\\Desktop\\BTS_1\\Training Data",
    target_size=(150, 150),
    batch_size=32,
    class_mode='categorical'
)

train_generator.class_indices

from tensorflow.keras.preprocessing.image import ImageDataGenerator

test_datagen = ImageDataGenerator(rescale=1./255) # Rescaling

# Make sure to replace 'path_to_your_test_directory' with your actual test data path
# Use double backslashes for Windows path or prefix the string with 'r' to avoid escape sequence errors
test_directory = r"C:\\Users\\dhars\\Desktop\\BTS_1\\Testing Data"

test_generator = test_datagen.flow_from_directory(
    directory=test_directory,
    target_size=(150, 150), # Adjust based on your model input size
    batch_size=20, # Can be adjusted based on your needs
    class_mode='categorical', # 'binary' if it's a binary classification
    shuffle=False) # Important for evaluation

# from tensorflow.keras.callbacks import EarlyStopping

## Define early stopping callback
# early_stopping = EarlyStopping(
#     monitor='val_loss',
#     patience=10,
#     restore_best_weights=True
# )

## Train the model with early stopping

```

```

# history = model.fit(
#     train_generator,
#     epochs=50, # Starting with 50 epochs as an example
#     validation_data=test_generator,
#     callbacks=[early_stopping],
#     verbose=1
# )

# eval_result = model.evaluate(test_generator)
# print(f'Test Loss: {eval_result[0]}, Test Accuracy: {eval_result[1]}")

# After training, save the model to an h5 file
# model.save('model_after_training.h5')

# model.save('C:\\Users\\dhars\\Desktop\\model_after_training.h5')

loaded_model = load_model('model_after_training.h5')

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense, Lambda
from tensorflow.keras import backend as K

def initialize_base_network(input_shape):
    """
    Base network to be shared (Siamese).
    """
    input = Input(shape=input_shape)
    x = Conv2D(64, (3, 3), activation='relu')(input)
    x = MaxPooling2D((2, 2))(x)
    x = Conv2D(128, (3, 3), activation='relu')(x)
    x = MaxPooling2D((2, 2))(x)
    x = Flatten()(x)
    x = Dense(128, activation='relu')(x)
    return Model(inputs=input, outputs=x)

# Define the tensors for the two input images
input_shape = (150, 150, 3)
input_a = Input(shape=input_shape)
input_b = Input(shape=input_shape)

# Create the base network
base_network = initialize_base_network(input_shape)

# Because we re-use the same instance `base_network`,
# the weights of the network will be shared across the two branches
processed_a = base_network(input_a)
processed_b = base_network(input_b)

# Compute the Euclidean distance between the two vectors in the latent space
distance = Lambda(lambda embeddings: K.sqrt(K.sum(K.square(embeddings[0] - embeddings[1]), axis=-1,
keepdims=True)))(
    [processed_a, processed_b])

# Add a dense layer with a sigmoid unit to generate the similarity score
similarity = Dense(1, activation='sigmoid')(distance)

# Define the model
model = Model(inputs=[input_a, input_b], outputs=similarity)

```

```

# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# Display the model summary
model.summary()

# Note: You would train this model by feeding pairs of images (input_a, input_b) and a label indicating if
they are similar (1) or not (0).

# Assuming you have a model defined something like this
model = Sequential([
    # Example convolutional base
    Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    MaxPooling2D(2, 2),
    # Add more layers as per your model

    Flatten(),
    # Before adding the dense layer, ensure the number of incoming features matches the dense layer's
    expected input
    Dense(512, activation='relu'), # This line might need adjustment
    Dense(7, activation='softmax')
])

# Print model summary to check the output shapes at each layer
model.summary()

def contrastive_loss(y_true, y_pred):
    """
    Contrastive loss function.
    :param y_true: True labels (1 for positive pairs, 0 for negative pairs)
    :param y_pred: Predicted distances
    :return: Contrastive loss value
    """
    margin = 1
    square_pred = K.square(y_pred)
    margin_square = K.square(K.maximum(margin - y_pred, 0))
    return K.mean(y_true * square_pred + (1 - y_true) * margin_square)

model.compile(loss=contrastive_loss, optimizer='adam', metrics=['accuracy'])

test_datagen = ImageDataGenerator(rescale=1./255)
test_generator = test_datagen.flow_from_directory(
    "C:\\Users\\dhars\\Desktop\\BTS_1\\Testing Data",
    target_size=(150, 150),
    batch_size=20, # Same as in model training if using batch-dependent layers
    class_mode='categorical',
    shuffle=False
)

# Predict the output
predictions = model.predict(test_generator)

# Convert predictions from one-hot to labels
predicted_classes = np.argmax(predictions, axis=1)

```

```

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Get the true labels from the generator
true_classes = test_generator.classes

# Generate the confusion matrix
cm = confusion_matrix(true_classes, predicted_classes)

class_labels = ['J Hope', 'Jimin', 'Jin', 'Jungkook', 'RM', 'Suga', 'V']

# Plot the confusion matrix
plt.figure(figsize=(10, 10))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_labels, yticklabels=class_labels)
plt.title('Confusion Matrix')
plt.ylabel('True Labels')
plt.xlabel('Predicted Labels')
plt.show()

from sklearn.metrics import classification_report

report = classification_report(true_classes, predicted_classes, target_names=class_labels)
print(report)

from sklearn.metrics import accuracy_score

accuracy = accuracy_score(true_classes, predicted_classes)
print(f'Model Accuracy: {accuracy:.2f}')

from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
import numpy as np

def predict_member(image_path, model):
    # Load the image file, targeting the size expected by the model
    img = image.load_img(image_path, target_size=(150, 150))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array /= 255.0 # Rescale the image (as done during training)

    # Predict the class
    prediction = model.predict(img_array)
    predicted_class = np.argmax(prediction, axis=1)[0]

    # Correct the class labels according to the class indices
    class_labels = ['J Hope', 'Jimin', 'Jin', 'Jungkook', 'RM', 'Suga', 'V']

    # Return the predicted class label and the corresponding probability
    return class_labels[predicted_class], np.max(prediction)

# Simulate uploading an image and making a prediction
uploaded_image_path = r"C:\Users\dhars\Desktop\New folder\OIP.jpeg"
predicted_member, probability = predict_member(uploaded_image_path, model)
print(f'Predicted member: {predicted_member}, Probability: {probability:.4f}')

# New Notebook

import os

```

```

import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.layers import Input, Conv2D, Lambda, Dense, Flatten, MaxPooling2D, Dropout
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import backend as K

# Step 1: Data Preparation - Function to Generate Image Pairs
def generate_image_pairs(image_directory, total_pairs=1000):
    categories = os.listdir(image_directory)
    categories = [c for c in categories if os.path.isdir(os.path.join(image_directory, c))]

    positive_pairs = []
    negative_pairs = []

    # Generate positive pairs
    for category in categories:
        path = os.path.join(image_directory, category)
        images = os.listdir(path)

        for i in range(int((total_pairs / (2 * len(categories))))):
            img1, img2 = np.random.choice(images, 2, replace=False)
            positive_pairs.append([os.path.join(path, img1), os.path.join(path, img2)], 1))

    # Generate negative pairs
    for i in range(int((total_pairs / 2))):
        category1, category2 = np.random.choice(categories, 2, replace=False)
        path1, path2 = os.path.join(image_directory, category1), os.path.join(image_directory, category2)
        img1, img2 = np.random.choice(os.listdir(path1), 1)[0], np.random.choice(os.listdir(path2), 1)[0]
        negative_pairs.append([os.path.join(path1, img1), os.path.join(path2, img2)], 0))

    pairs = positive_pairs + negative_pairs
    np.random.shuffle(pairs)

    return pairs

# Step 2: Preprocessing Image Pairs
def preprocess_pairs(pairs, target_size=(224, 224)):
    pair1 = []
    pair2 = []
    labels = []

    for pair, label in pairs:
        img1 = tf.keras.preprocessing.image.load_img(pair[0], target_size=target_size)
        img2 = tf.keras.preprocessing.image.load_img(pair[1], target_size=target_size)

        img1 = tf.keras.preprocessing.image.img_to_array(img1)
        img2 = tf.keras.preprocessing.image.img_to_array(img2)

        img1 /= 255.0
        img2 /= 255.0

        pair1.append(img1)
        pair2.append(img2)
        labels.append(label)

    return np.array(pair1), np.array(pair2), np.array(labels)

```

```

# Siamese Network Model Creation
def create_base_network(input_shape):
    base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
    for layer in base_model.layers:
        layer.trainable = False
    model = Sequential([
        base_model,
        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(128, activation='relu'),
        Dropout(0.5)
    ])
    return model

input_shape = (224, 224, 3)
base_network = create_base_network(input_shape)

input_a = Input(shape=input_shape)
input_b = Input(shape=input_shape)

processed_a = base_network(input_a)
processed_b = base_network(input_b)

distance = Lambda(lambda x: K.abs(x[0] - x[1]))([processed_a, processed_b])
outputs = Dense(1, activation='sigmoid')(distance)
model = Model([input_a, input_b], outputs)

model.compile(loss='binary_crossentropy', optimizer=Adam(0.0001), metrics=['accuracy'])

# Model Training
def train_model(image_directory, total_pairs=1000, epochs=10):
    pairs = generate_image_pairs(image_directory, total_pairs)
    pair1, pair2, labels = preprocess_pairs(pairs)
    history = model.fit([pair1, pair2], labels, epochs=epochs, batch_size=32)
    return history

# Prediction Function
def predict_member_similarity(model, img_path1, img_path2, target_size=(224, 224)):
    img1 = tf.keras.preprocessing.image.load_img(img_path1, target_size=target_size)
    img2 = tf.keras.preprocessing.image.load_img(img_path2, target_size=target_size)

    img1 = tf.keras.preprocessing.image.img_to_array(img1) / 255.0
    img2 = tf.keras.preprocessing.image.img_to_array(img2) / 255.0

    pred = model.predict([np.array([img1]), np.array([img2])])
    return pred[0][0]

from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping

# Callbacks
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, min_lr=1e-7, verbose=1)
early_stopping = EarlyStopping(monitor='val_loss', patience=5, verbose=1)

# Assuming you have a directory with training images organized in subdirectories for each class
image_directory = "C:\\Users\\dhars\\Desktop\\BTS_1\\Training Data"

# Generate image pairs
pairs = generate_image_pairs(image_directory)

```

```

# Preprocess image pairs
pair1, pair2, labels = preprocess_pairs(pairs)

# Now proceed to training with these variables
# Reduced epochs and increased batch size for quicker training
history = model.fit(
    [pair1, pair2], labels,
    validation_split=0.1,
    epochs=70, # Reduced from 70 to 20
    batch_size=32, # Increased from 32 to 64
    callbacks=[reduce_lr, early_stopping]
)

# Assuming you have a function or process to generate test pairs similar to the training pairs
test_pairs = generate_image_pairs("C:\\Users\\dhars\\Desktop\\BTS_1\\Testing Data", total_pairs=200)
test_pair1, test_pair2, test_labels = preprocess_pairs(test_pairs)

# Evaluate the model on the test set
evaluation = model.evaluate([test_pair1, test_pair2], test_labels)
print(f"Test Loss: {evaluation[0]}, Test Accuracy: {evaluation[1]}")

# Example usage of the prediction function
similarity_score = predict_member_similarity(model, "C:\\Users\\dhars\\Desktop\\Reference Images\\Jin\\member.jpg", "C:\\Users\\dhars\\Desktop\\New folder\\bhbhlkbbh.jpeg")
print(f"Similarity Score: {similarity_score}")

# # Save the model
# model.save("C:\\Users\\dhars\\Desktop\\bts_siamese_model.h5")

# Load the model
model = tf.keras.models.load_model('bts_siamese_model.h5')

from sklearn.metrics import classification_report, confusion_matrix

# Predictions
predictions = model.predict([test_pair1, test_pair2])
predicted_labels = [1 if pred > 0.5 else 0 for pred in predictions]

# True labels
true_labels = test_labels

# Classification report
print(classification_report(true_labels, predicted_labels))

# Confusion matrix
conf_matrix = confusion_matrix(true_labels, predicted_labels)
print(conf_matrix)

import ipywidgets as widgets
from IPython.display import display, Image
from tensorflow.keras.preprocessing import image as keras_image
import os
import numpy as np

# Load the model
model = tf.keras.models.load_model('bts_siamese_model.h5')

# Directory containing one reference image per BTS member
reference_images_dir = 'C:\\Users\\dhars\\Desktop\\Reference Images\\Jin'

```

```

# Load reference images and preprocess them
reference_images = {}
for filename in os.listdir(reference_images_dir):
    member_name = os.path.splitext(filename)[0] # Assuming the filename is the member's name
    img_path = os.path.join(reference_images_dir, filename)
    img = keras_image.load_img(img_path, target_size=(224, 224))
    img_array = keras_image.img_to_array(img) / 255.0
    reference_images[member_name] = img_array

# Define the prediction function
def predict_member(upload_image_array):
    highest_similarity = 0
    predicted_member = None

    # Compare the uploaded image with each reference image
    for member, ref_img_array in reference_images.items():
        similarity = model.predict([np.array([upload_image_array]), np.array([ref_img_array])])[0][0]
        if similarity > highest_similarity:
            highest_similarity = similarity
            predicted_member = member

    return predicted_member, highest_similarity

# Upload button
uploader = widgets.FileUpload()
display(uploader)

# Prediction button
predict_button = widgets.Button(description="Predict Member")
display(predict_button)

def on_predict_button_clicked(b):
    if uploader.value:
        # Get the uploaded file data
        # Since uploader.value is misbehaving, access the data directly
        uploaded_file = list(uploader.value.items())[0]
        file_name, file_info = uploaded_file
        content = file_info['content']
        with open("uploaded_image.jpeg", "wb") as f:
            f.write(content)

        # Display the uploaded image
        display(Image("uploaded_image.jpeg"))

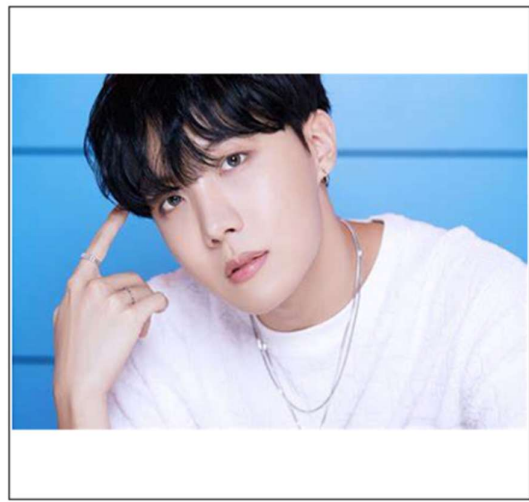
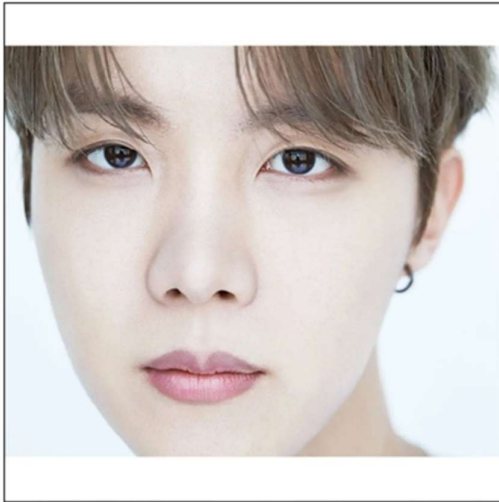
        # Preprocess the image and make predictions
        img = keras_image.load_img("uploaded_image.jpeg", target_size=(224, 224))
        img_array = keras_image.img_to_array(img) / 255.0

        predicted_member, confidence = predict_member(img_array)
        print(f"Predicted Member: {predicted_member} with confidence: {confidence:.2%}")
    else:
        print("Please upload an image.")

# Connect the button to the function
predict_button.on_click(on_predict_button_clicked)

```

SCREENSHOTS:



1/1 [=====] - 0s 491ms/step
Predicted BTS member: J Hope

1/1 [=====] - 0s 279ms/step
Confidence score:0.47

9.2 PLAGIARISM REPORT

10 REFERENCES

1. Kumar, C. R., N, S., Priyadharshini, M., E, D. G., & M, K. R. (2023). Face recognition using CNN and Siamese network. *Measurement: Sensors*, 27(2023), 100800.
2. Solomon, E., Woubie, A., & Emiru, E. S. (2024). Deep Learning Based Face Recognition Method using Siamese Network. *arXiv preprint arXiv:2312.14001*.
3. Hamdani, N. D., Bousahba, N., Bousbai, A. H., & Braikia, A. (2023). Face Detection and Recognition Using Siamese Neural Network. *International Journal of Computing and Digital Systems*, 14(1), 889-897.
4. Yu, H. Q. (2024). Attention Enhanced Siamese Neural Network for Face Validation. *Artificial Intelligence and Applications*, 2(1), 21–27.
5. Navghare, T., Muley, A., & Jadhav, V. (2024). Siamese Neural Networks for Kinship Prediction: A Deep Convolutional Neural Network Approach. *Indian Journal of Science and Technology*, 17(4), 352-358.
6. Menendez, E., Martínez, S., Díaz-de-María, F., & Balaguer, C. (2024). Integrating Egocentric and Robotic Vision for Object Identification Using Siamese Networks and Superquadric Estimations in Partial Occlusion Scenarios. *Biomimetics*, 9(100), 1-21.
7. Hajamohideen, F., Shaffi, N., Mahmud, M., Subramanian, K., Al Sariri, A., Vimbi, V., ... & for the Alzheimer's Disease Neuroimaging Initiative. (2023). Four-way classification of Alzheimer's disease using deep Siamese convolutional neural network with triplet-loss function. *Brain Informatics*, 10(5).